

Teste Técnico — Desenvolvedor(a) Full-Stack

Mini-CRM WhatsApp · E3

Contexto

A E3 está construindo um CRM integrado ao WhatsApp para atendimento multi-unidade. Este teste simula um recorte real do produto: conectar um número de WhatsApp, responder mensagens automaticamente e controlar o acesso por unidade de negócio.

Objetivo

Construir um mini-CRM funcional com integração WhatsApp, autenticação via Google e isolamento de dados por unidade.

Requisitos funcionais

1. Integração WhatsApp

- Conectar uma instância de WhatsApp. Bibliotecas/abordagens aceitas: **Baileys**, **whatsapp-web.js**, **Evolution API** ou **WhatsApp Cloud API** (oficial).
- Se usar biblioteca não-oficial: exibir o **QR Code de pareamento** no frontend ou no log do container.
- **Bot de auto-resposta**: ao receber a mensagem Oi, responder exatamente:
 “Oi! Aqui é o Atendente da E3”
- Persistir as mensagens recebidas em banco de dados, associadas à unidade dona da conexão.

2. Autenticação e multi-unidade

- Login com **Google** (Firebase Authentication).
- Cada usuário pertence a uma **unidade** (filial). O vínculo usuário→unidade pode ser feito via seed/painel simples.
- Usuário autenticado enxerga **apenas os dados da própria unidade** (mensagens, conversas).
- Demonstrar com no mínimo **2 unidades** e 1 usuário em cada.

3. Frontend

- Tela de login (Google Sign-In).
- Tela listando mensagens/conversas da unidade do usuário logado.
- **Deploy no Firebase Hosting** — a URL pública faz parte da entrega.

4. Backend

- API que serve o frontend (REST ou tRPC/GraphQL, à sua escolha).
- Serviço de conexão com o WhatsApp.
- Banco de dados de sua escolha — justificar a escolha no README.

Requisitos técnicos obrigatórios

Item	Detalhe
Monorepo	Estrutura apps/web, apps/api (+ packages/ compartilhados se fizer sentido). Ferramenta livre: pnpm workspaces, Turborepo, Nx.
Docker	Backend + banco orquestrados via docker - compose. Um único docker compose up deve subir o ambiente local.

Item	Detalhe
Commits atômicos	O histórico Git será avaliado. Um commit = uma mudança lógica, com mensagem descritiva (padrão Conventional Commits é bem-visto). Não serão aceitos commits únicos do tipo “projeto completo”.
README	Setup local passo a passo, decisões de arquitetura (2–3 parágrafos), URL do deploy, e o que faria diferente com mais tempo.

Entrega

- **Prazo: 72 horas** a contar do recebimento deste briefing.
- Link do repositório Git (público, ou acesso concedido ao avaliador).
- URL do frontend publicado no Firebase Hosting.
- *(Opcional, conta pontos)* vídeo de até 5 minutos demonstrando o fluxo completo: login → QR Code → envio de “Oi” → resposta automática → mensagem aparecendo no CRM.

Critérios de avaliação

Critério	Peso
Funcionalidade (bot WhatsApp + auth Google + isolamento por unidade)	35%
Arquitetura e organização do monorepo	20%
Docker e reprodutibilidade do ambiente	15%
Qualidade do histórico de commits	15%
Documentação (README)	10%
Deploy funcional no Firebase Hosting	5%

O que NÃO é exigido

- UI elaborada — funcional e limpa basta.
- Cobertura extensa de testes (testes nos pontos críticos contam pontos, mas não são obrigatórios).
- Envio ativo de mensagens/campanhas — apenas a auto-resposta descrita.
- CI/CD — deploy manual no Firebase Hosting é aceito.

Observações

- Bibliotecas não-oficiais de WhatsApp violam os Termos de Serviço do WhatsApp — use um número descartável/secundário para o teste.
- Em caso de dúvida sobre escopo, envie a pergunta ao avaliador; saber perguntar também é avaliado.

E3 · Processo seletivo — uso interno RH e candidatos